

M5 – Reporting Module

Final report:

Purpose: This is a detailed handover of my M5 Reporting work for the team member integrating the modules. It records exactly what was created, where it is located in GitHub, what the current code does, what it does not do yet, and what needs to happen during database/integration work.

1. Quick Status

Area	Current status	Notes
M5 folder structure	Done	Dedicated Reporting folder in Application and 05_M5_Reporting in Web Controllers.
Reporting DTOs	Done	Four report DTOs created.
ReportingService	Scaffold done	Service exists; current statistics method uses placeholders.
ReportingController	Scaffold done	Reporting endpoints are defined; list endpoints currently return empty collections.
Database integration	Pending	Depends on shared persistence/database layer.
Actual reporting queries	Pending	Replace placeholders with real data access/calculations.
Recurring incident analysis	Pending	Not present in the current M5 code.
Build	Successful	Full solution built successfully after the M5 files were added.
GitHub	Pushed	Committed and pushed to main.

2. Repository / GitHub Location

- Repository: FlameTS/AI-ITSM
- Branch: main
- Commit: f9b2cfb – Add M5 reporting module scaffold
- Push result: the commit was successfully pushed to origin/main.

3. Exact GitHub File Structure

The following six files are the M5 files added in my contribution:

- Source/AIITSM/AIITSM.Application/Reporting/EscalatedIncidentDto.cs

- Source/AIITSM/AIITSM.Application/Reporting/IncidentStatisticsDto.cs
- Source/AIITSM/AIITSM.Application/Reporting/ReportingService.cs
- Source/AIITSM/AIITSM.Application/Reporting/SupportTeamPerformanceDto.cs
- Source/AIITSM/AIITSM.Application/Reporting/UnresolvedIncidentDto.cs
- Source/AIITSM/AIITSM.Web/Controllers/05_M5_Reporting/ReportingController.cs

4. What Each File Contains

IncidentStatisticsDto.cs

Namespace: AIITSM.Application.Reporting.

Properties:

- TotalIncidents
- OpenIncidents
- ResolvedIncidents
- EscalatedIncidents

Purpose: defines the shape of the overall incident statistics response. It does not query the database itself.

UnresolvedIncidentDto.cs

Properties:

- IncidentId
- Title
- Priority
- Status
- CreatedAt

Purpose: defines the data returned for an unresolved incident. It is a data contract, not a database query.

EscalatedIncidentDto.cs

Properties:

- IncidentId
- Title
- Priority
- Status
- EscalatedBy (nullable int)
- EscalatedTo (nullable int)
- Reason
- EscalatedAt

Purpose: defines the data required for an escalated-incident report, including escalation information.

SupportTeamPerformanceDto.cs

Properties:

- UserId
- UserName
- AssignedIncidents
- ResolvedIncidents
- OpenIncidents

Purpose: defines the output structure for support-team/user performance information.

ReportingService.cs

Namespace: AllTSM.Application.Reporting.

Contains ReportingService and GetIncidentStatistics().

Current behavior:

GetIncidentStatistics() returns an IncidentStatisticsDto with all four counts set to 0.

Important: these are placeholders. There is no database query in the current service. The service is the intended Application-layer location for the reporting logic once the shared data/persistence layer is available.

ReportingController.cs

Located in the Web layer under Controllers/05_M5_Reporting.

The controller exposes the M5 reporting operations. The current code includes operations for:

- incident statistics
- unresolved incidents
- escalated incidents
- team performance

The current list operations return empty DTO collections because no live database/persistence source is connected yet.

5. Current Reporting Flow

The intended responsibility split is:

- Web layer → ReportingController receives/exposes reporting requests.
- Application layer → ReportingService contains the reporting/application logic.
- Application DTOs → define the structured output expected by each report.
- Infrastructure/shared persistence → should provide access to actual database data when integrated.
- Database/domain data → supplies incidents, users, assignments and escalation information needed to produce real report results.

6. What the Integrator Needs to Do Later

1. Connect ReportingService to the project's existing/shared persistence or repository mechanism rather than creating an unrelated database connection specifically for M5.
2. Use the existing incident/user/assignment/escalation data structures from the rest of the project.
3. Replace the placeholder zero values in GetIncidentStatistics() with actual database-backed calculations.
4. Replace the empty collections returned by the list operations with actual data retrieval and mapping into the corresponding DTOs.
5. Implement the remaining reporting requirements that are not present in the current scaffold, including recurring incident pattern analysis if it is part of the team's assigned M5 scope.
6. Test the reporting operations using actual incident data.
7. If a UI/dashboard is required by the final project, connect the reporting API responses to the relevant presentation layer.

7. Data the Reporting Module Will Need

Based on the DTOs already created, the reporting layer will need access to information such as:

- Incident ID, title, priority and status.
- Incident creation time.
- Assignment/user information.
- Escalation information, including escalated-by, escalated-to, reason and time.
- Incident status for calculating open/resolved counts.

The exact table/entity/repository names should be taken from the team's shared database/domain implementation. They should not be assumed from this scaffold.

8. What Has NOT Been Done in M5

- No M5-specific database connection was created.
- No actual SQL/EF Core reporting queries were added.
- No fake database data was added.
- GetIncidentStatistics() currently returns zeros.
- The collection-based controller operations currently return empty arrays.
- Recurring incident pattern analysis is not implemented in the current files.
- No claim should be made that live reporting is already working.

9. Build / Verification

After creating the M5 files, the full solution was built successfully. The build output showed AIITSM.Domain, AIITSM.Application, AIITSM.Infrastructure and AIITSM.Web all succeeding.

This confirms that the current M5 scaffold is compiling with the existing solution.

10. Git History / Contribution Proof

The M5 contribution was staged, committed and pushed to the team's GitHub repository.

- Commit message: Add M5 reporting module scaffold
- Commit: f9b2cfb
- Branch: main
- Remote: origin
- Push completed successfully to origin/main.

11. Integration Precautions

- Do not duplicate the database layer inside M5 if the team already has a shared Infrastructure/persistence implementation.
- Keep DTOs as response/data-contract classes; database access should not be placed inside the DTOs.
- Keep HTTP/API concerns in ReportingController and reporting/application logic in ReportingService.
- Use the project's actual domain/entity/property names when implementing queries. The current DTO property names describe the required report output but do not prove the database schema.
- Before changing the controller method signatures, coordinate with whoever is integrating the Web/API layer so existing routes are not broken.

12. Final Handover Summary

M5 Reporting has been structurally set up and pushed to GitHub. The current contribution establishes the Application-layer reporting contracts, the ReportingService boundary, and the Web-layer controller. The implementation intentionally stops before database-dependent logic. The integrator can use this document to locate the files and continue the database-backed implementation without recreating the M5 structure.